Fundamentals of Cybersecurity

Mohammad Hosseini

Faculty of Computer Science and Engineering
Shahid Beheshti University

Network Security Countermeasures 1



Outline

This chapter up to this point: Threats and Attacks in the Network

In the rest of this chapter, we consider three categories of **controls**:

- Encryption
  - We describe architecturally how encryption can be used
  - We introduce two specific applications of cryptography to networking: SSL and VPN
- Firewall
- Intrusion detection or protection system

Cryptography in Network Security

Network encryption

Before we begin to study the use of encryption to counter network security threats, we should consider these points.

- Encryption protects only what is encrypted (*which should be obvious but isn't always*).
- Designing encryption algorithms is best left to professionals.
- Encryption is no more secure than its key management.
- Encryption is not a silver bullet. A flawed system design with encryption is still a flawed system design.

Modes of network encryption

In network applications, encryption can be applied either

- between two hosts (called link encryption)

  or

- between two applications (called end-to-end encryption).

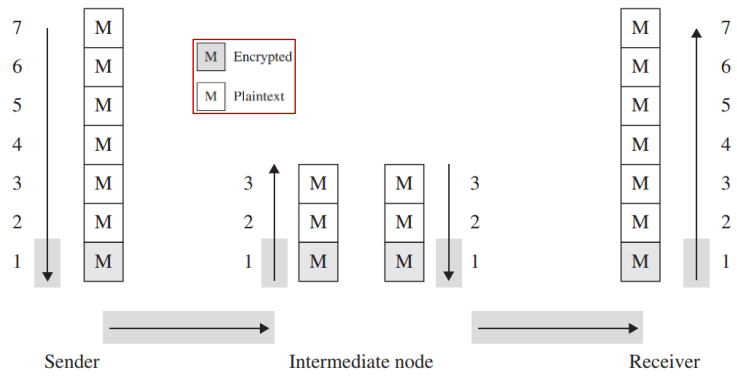
➢ They perform different functions and have different strengths and weaknesses.

Link encryption

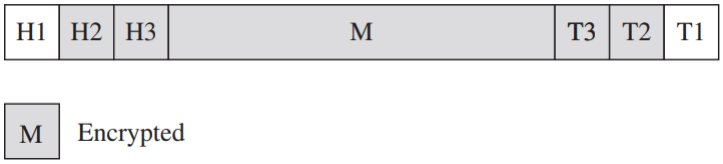
Link encryption: data are encrypted just before the system places them on the physical communications link.

- Encryption occurs at layer 1 or 2 in the OSI model.
- Decryption occurs just as the communication arrives at and enters the receiving computer.

➢ Link encryption covers a communication from one node to the next on the path to the destination.



Link encryption



- Encryption protects the message in transit between two computers, but the message is in plaintext inside the hosts.
  - Because the encryption is added at the bottom protocol layer, the message is exposed in all other layers of the sender and receiver.

Link encryption

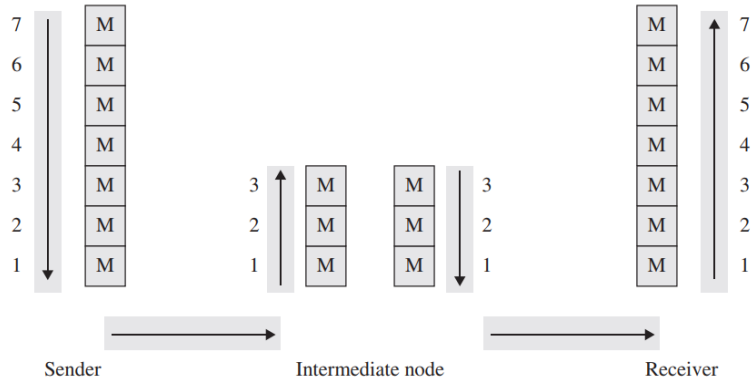
- Hardware encryption devices operate quickly and reliably.
  - In this case, link encryption is invisible to the operating system as well as to the operator.
- Link encryption is especially appropriate when the transmission line is the point of greatest vulnerability.
- If all hosts on a network are reasonably secure but the communications medium is shared with other users or is not secure, link encryption is an easy control to use.

End-to-end encryption

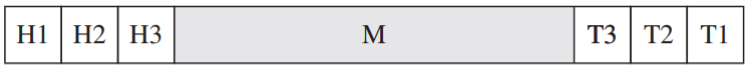
End-to-end encryption: provides security from one end of a transmission to the other.

- The encryption is performed at the highest levels, usually by an application at OSI level 7, but sometimes 5 or 6.

- End-to-end encryption covers a communication from origin to destination.
- Even though a message must pass through potentially insecure nodes, the message is protected against disclosure while in transit.



End-to-end encryption



M Encrypted

- Because the encryption precedes all the routing and transmission processing of the layer, the message is transmitted in encrypted form throughout the network.
- Only the **data portion** of the message is protected, but often the headers are not as sensitive as the data.
- Each pair of users should share a unique cryptographic key.
- Since the intermediate hosts along a transmission path do not need to encrypt or decrypt the message traffic, they have no need for cryptographic facilities.

Transport Layer Security (TLS)

The need for TLS in Internet browsing

Bob is surfing the Web and arrives at the Alice Incorporated site, which is selling perfume …

- If no confidentiality (encryption) is used, an intruder could intercept Bob's order and obtain his payment card information.
- If no data integrity is used, an intruder could modify Bob's order or address.
- If no server authentication is used, a server could display Alice Incorporated's famous logo when in actuality the site maintained by Trudy, who is masquerading as Alice Incorporated.
  - After receiving Bob's order, Trudy could take Bob's money and run,
  - or Trudy could carry out an identity theft by collecting Bob's name, address, and credit card number.

Transport Layer Security (TLS)

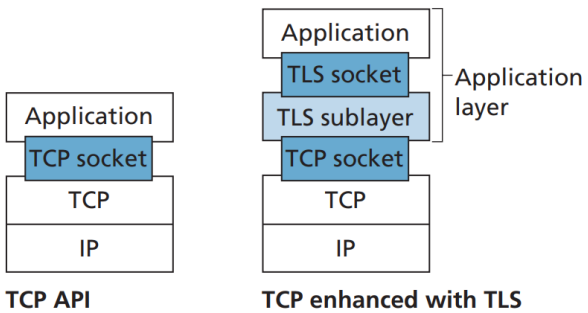
- TLS enhances TCP with confidentiality, data integrity, server authentication, and client authentication.
- The earlier version of this protocol is SSL (Secure Sockets Layer)
  - The SSL protocol was originally designed by Netscape in the mid-1990s to protect communication between a web browser and server.
  - SSL 2.0 (1995), SSL 3.0 (1996)



- In 1999, the Internet Engineering Task Force (IETF) upgraded SSL 3.0 and named the upgrade TLS.
  - TLS 1.1 (RFC 4346 in 2006), TLS 1.2 (RFC 5246 in 2008), TLS 1.3 (RFC 8446 in 2018)
  - ✓ The acronym SSL is often used to refer to both the SSL and TLS protocol suites.
- TLS is supported by all popular Web browsers and Web servers.

TLS

- TLS is often used to provide security to transactions that take place over HTTP.
- However, because TLS secures TCP, it can be employed by any application that runs over TCP.
- TLS provides a simple Application Programmer Interface (API) with sockets, which is similar and analogous to TCP's API.
- When an application wants to employ TLS, the application includes SSL classes/libraries.



The big picture: almost-TLS

- We begin by describing a simplified version of TLS, one that will allow us to get a big-picture understanding of the why and how of TLS.
  - We refer to this simplified version of TLS as "*almost-TLS*."
  - After describing *almost-TLS*, we will then describe the real TLS.
- TLS has three phases: handshake, key derivation, and data transfer
- We describe these three phases for a communication session between a **client (Bob)** and a **server (Alice)**, with Alice having a private/public key pair and a certificate that binds her identity to her public key.

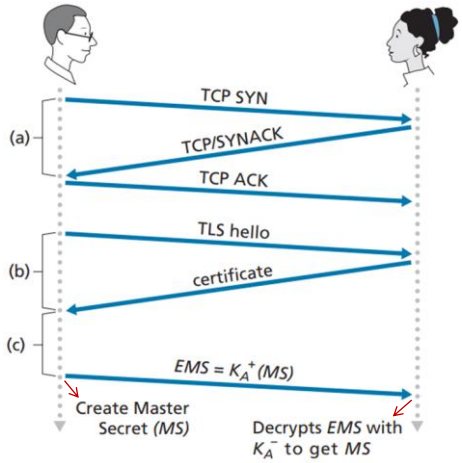
Almost-TLS: handshake

During the handshake phase, Bob needs to

- establish a TCP connection with Alice,
- verify that Alice is really Alice
- send Alice a master secret key
  - The **master secret key** will be used by both Alice and Bob to generate all the symmetric keys they need for the TLS session.

Almost-TLS: handshake

- Once the TCP connection is established, Bob sends Alice a `hello` message.
- Alice then responds with her certificate, which contains her public key.
- Because the certificate has been certified by a CA, Bob knows for sure that the public key in the certificate belongs to Alice.
- Bob then generates a Master Secret (MS), encrypts the MS with **Alice's public key** to create the Encrypted Master Secret (**EMS**), and sends the EMS to Alice.
- Alice decrypts the EMS with her private key to get the MS.
- After this phase, both Bob and Alice (and no one else) know the master secret for this TLS session.



Almost-TLS: key derivation

In principle, the **MS** could be used as the symmetric session key for all subsequent encryption and data integrity checking.

However, it is generally considered safer for Alice and Bob to each use **different** cryptographic keys, and also to use different keys for encryption and integrity checking.

- Thus, both Alice and Bob use the **MS** to generate **four keys**:
  - E_B: session encryption key for data sent from Bob to Alice
  - M_B: session HMAC key for data sent from Bob to Alice
  - E_A: session encryption key for data sent from Alice to Bob
  - M_A: session HMAC key for data sent from Alice to Bob
- This could be done by simply slicing the MS into four keys. (but in reality TLS it is a little more complicated)

Almost-TLS: data transfer

- Now that Alice and Bob share the same four session keys they can start to send secured data to each other over the TCP connection.
- Since TCP is a byte-stream protocol, a natural approach would be for TLS to encrypt application data on the fly and pass it to TCP.
  - Where should we put the HMAC for the integrity check?
    - We certainly do not want to wait until the end of the TCP session to verify the integrity of all of Bob's data that was sent over the entire session.

Almost-TLS: data transfer

- TLS breaks the data stream into records,
- appends an **HMAC** to each record for integrity checking,
- and then encrypts the **record+HMAC**.
- This encrypted package is then passed to TCP for transport over the Internet.

To create the **HMAC**, Bob inputs the record data along with the key M_B into a hash function.

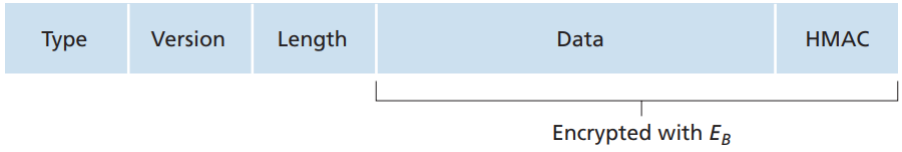
To encrypt the package **record+HMAC**, Bob uses his session encryption key E_B.

Is *almost-TLS* secure?

Man-in-the-middle

- Trudy, a woman-in-the-middle, may capture two segments sent by Bob, reverse the order of the segments, adjust the TCP sequence numbers, and then send the two reverse-ordered segments to Alice.
- Solution: sequence number
  - Bob maintains a sequence number counter, which begins at zero and is incremented for each TLS record he sends.
  - The sequence number is not included in the record itself.
  - When Bob calculates the HMAC, he includes the sequence number in the HMAC calculation → `HMAC = hash(data + M_B + current_sequence_number)`
  - Alice tracks Bob's sequence numbers

TLS record



- The first three fields are not encrypted.
- The type field indicates whether the record is a handshake message, a message that contains application data, or a TLS connection close record.
- TLS at the receiving end uses the length field to extract the TLS records out of the incoming TCP byte stream.

The real TLS

- SSL does not mandate that Alice and Bob use a specific symmetric key algorithm or a specific public-key algorithm.
  - TLS allows Alice and Bob to agree on the cryptographic algorithms during the handshake phase.
- During the handshake phase, Alice and Bob send nonces to each other, which are used in the creation of the session keys (E_B, M_B, E_A, and M_A)

The real TLS handshake

1. The client sends a list of cryptographic algorithms it supports, along with a client nonce (in `Client_Hello` message).
2. From the list, the server chooses a symmetric algorithm (for example, AES) and a public key algorithm (for example, RSA with a specific key length), and HMAC algorithm (MD5 or SHA-1).
   - It sends back to the client its choices, as well as a certificate and a server nonce (in `Server_Hello` message).
3. The client verifies the certificate, extracts the **server's public key**.
   - It generates a Pre-Master Secret (PMS), encrypts the **PMS** with the server's public key, and sends the encrypted **PMS** to the server.

The real TLS handshake

4. Using the same key derivation function (as specified by the TLS standard), the client and server independently compute the Master Secret (MS) from the **PMS** and nonces.
   - The MS is then sliced up to generate the two encryption and two HMAC keys.
   - Henceforth, all messages sent between client and server are encrypted and authenticated
5. The client sends the HMAC of all the handshake messages.

Why?

6. The server sends the HMAC of all the handshake messages.

TLS connection closure

- How to close a TLS connection?
- Is it a good idea to have Bob send a TCP **`FIN`** segment to Alice?
  - It sets the stage for the truncation attack whereby Trudy once again gets in the middle of an ongoing TLS session and ends the session early with a TCP **`FIN`**.
  - Alice thinks she has received all of Bob's data when actuality she has only received a portion of it.

❖ The solution to this problem is to indicate in the **type field** whether the record serves to terminate the TLS session.

Cipher suite

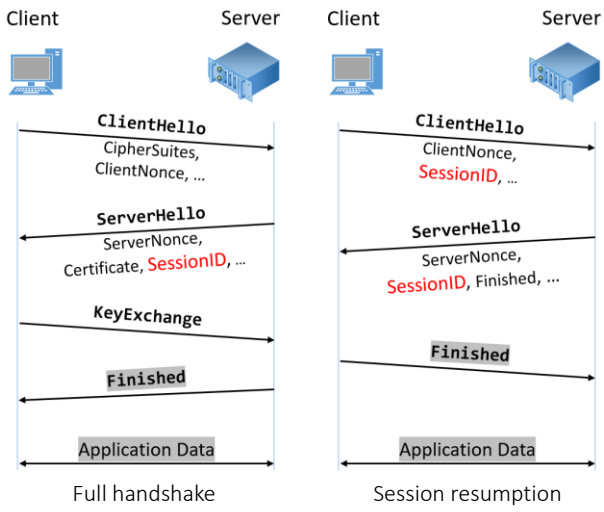
- A cipher suit is a set of cryptographic algorithms used to secure communications between a client and a server.
- During TLS handshaking, the client and server negotiate which cipher suite to use.
- The set of algorithms that cipher suites usually contain include:
  - Key exchange algorithm (e.g., RSA, ECDH, DHE)
  - Bulk encryption algorithm (e.g., 3DES, AES)
  - Authentication Algorithm (e.g., RSA, ECDSA)
  - Message authentication code algorithm (HMAC with SHA-256 or 384)

Cipher suite

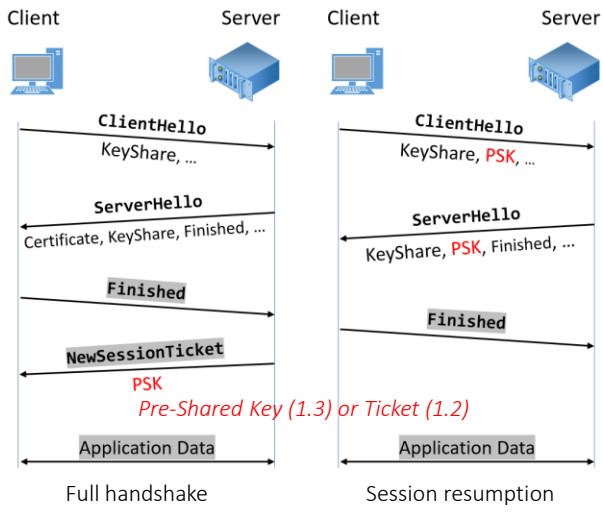
TLS 1.3 Cipher Suites

Cipher Suite Identifier	Algorithms Used
TLS_AES_256_GCM_SHA384	AES with a 256-bit key encryption, 384-bit SHA hash function
TLS_CHACHA20_POLY1305_SHA256	Chacha20–Poly1305 stream cipher, 256-bit SHA hash function
TLS_AES_128_GCM_SHA256	AES with a 128-bit key encryption, 256-bit SHA hash function
TLS_AES_128_CCM_8_SHA256	AES with a 128-bit key encryption, 256-bit SHA hash function
TLS_AES_128_CCM_SHA256	AES with a 128--bit key, SHA-1 hash function

TLS session resumption



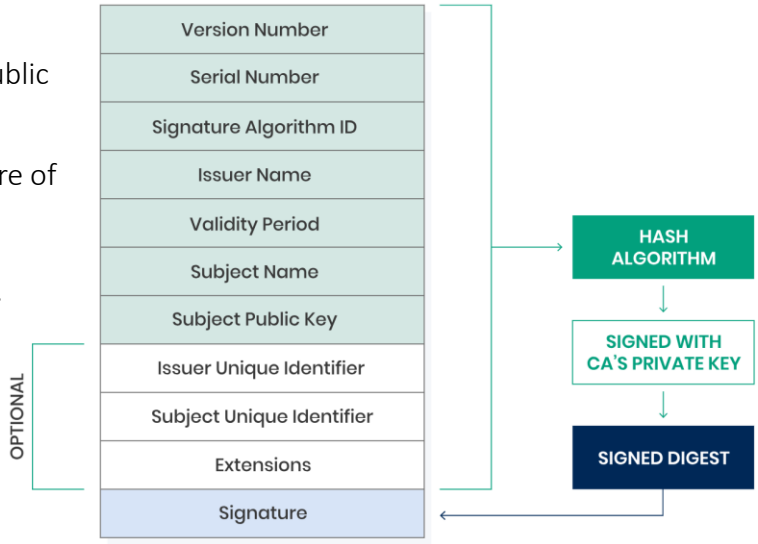
TLS 1.1, 1.2



TLS 1.2, 1.3

Certificate

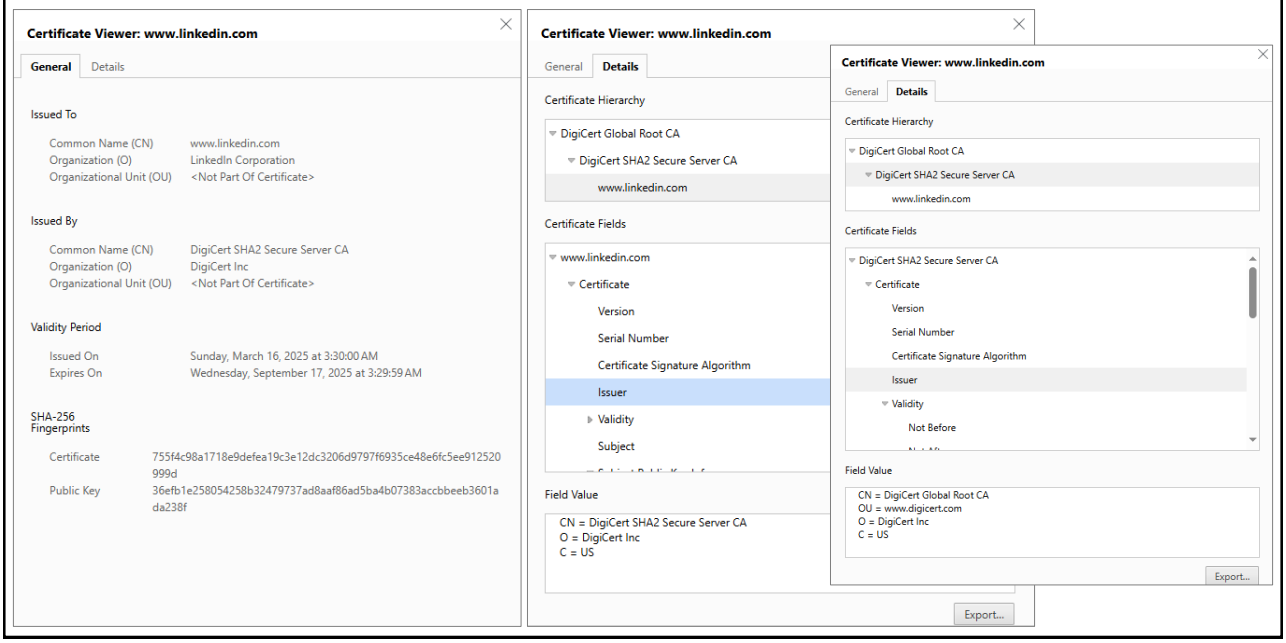
- X.509 is an International Telecommunication Union (ITU) standard defining the format of public key certificates.
- It specifies the format and structure of certificates used within Public Key Infrastructure (PKI) to secure communications and transactions.



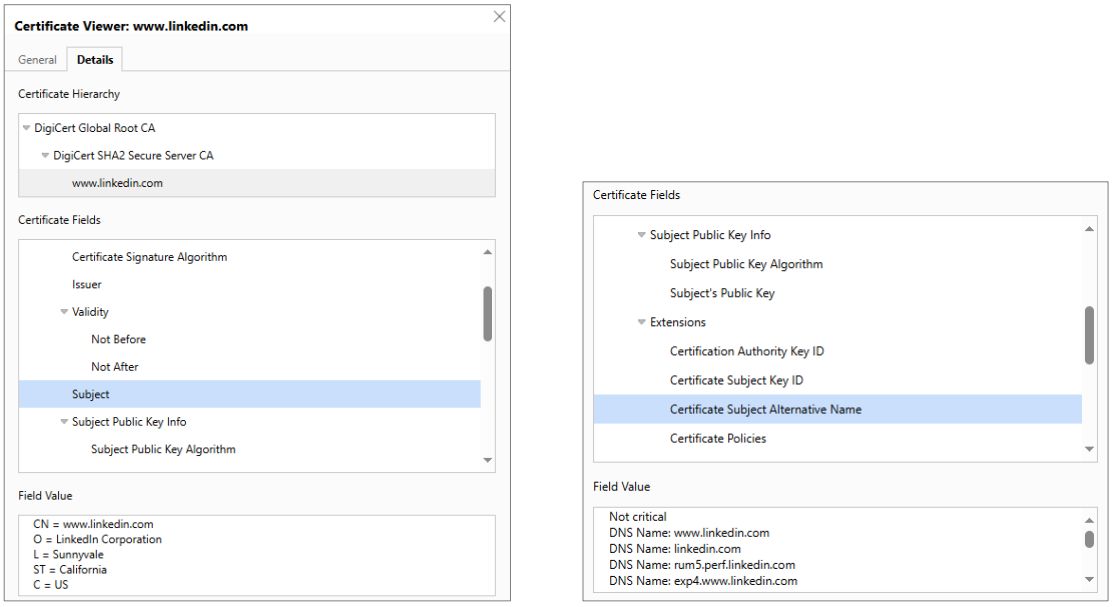
Certificate

- Every certificate is part of a chain of trust, starting from a trusted root certificate authority (CA) and ending with the leaf certificate (the one being verified).
- The chain of trust is established as follows:
  - **Root CA Certificate**: A trusted certificate that is self-signed and pre-installed in clients (browsers, operating systems, etc.).
  - **Intermediate CA Certificate(s)**: Issued by the Root CA to delegate signing authority to another CA.
  - **Leaf Certificate**: The certificate being verified, issued by the Intermediate CA.

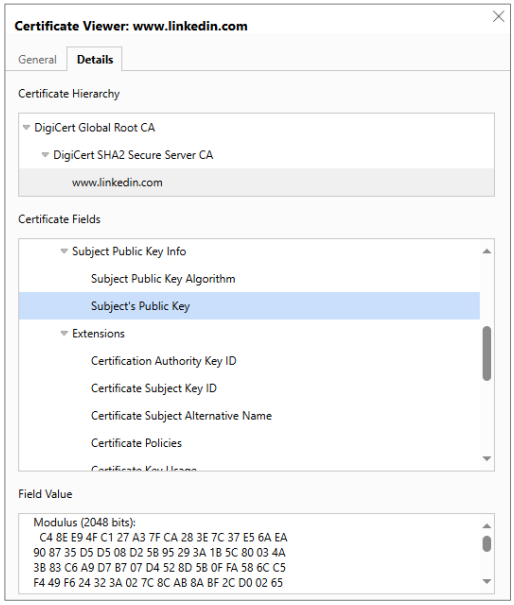
Certificate

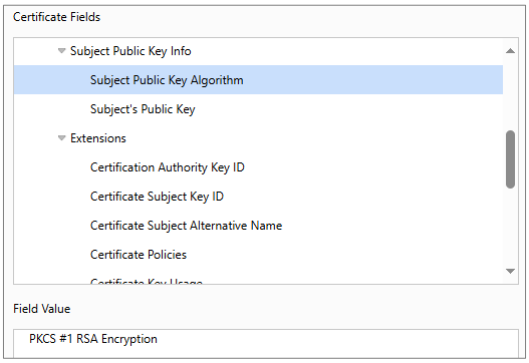


Certificate

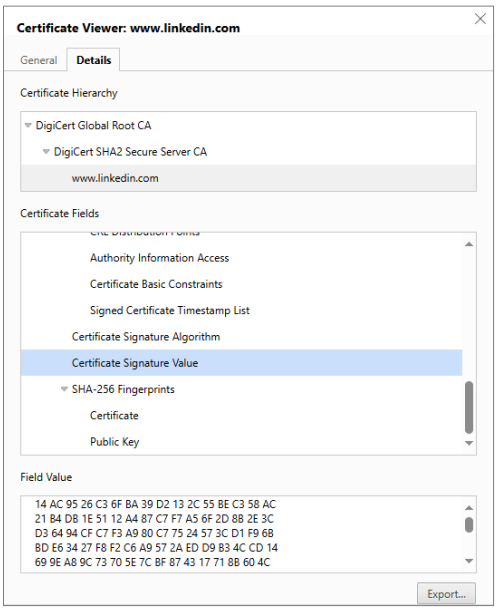


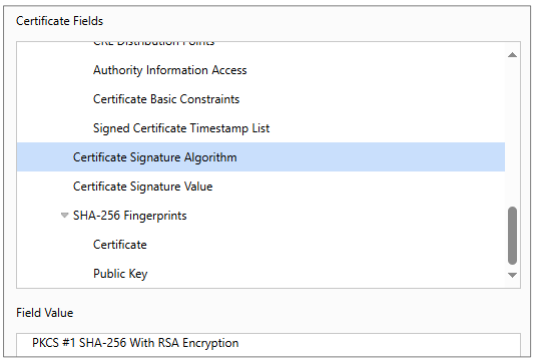
Certificate



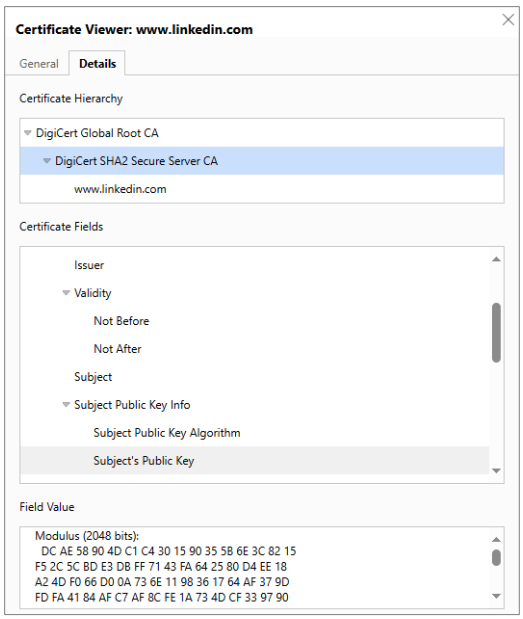


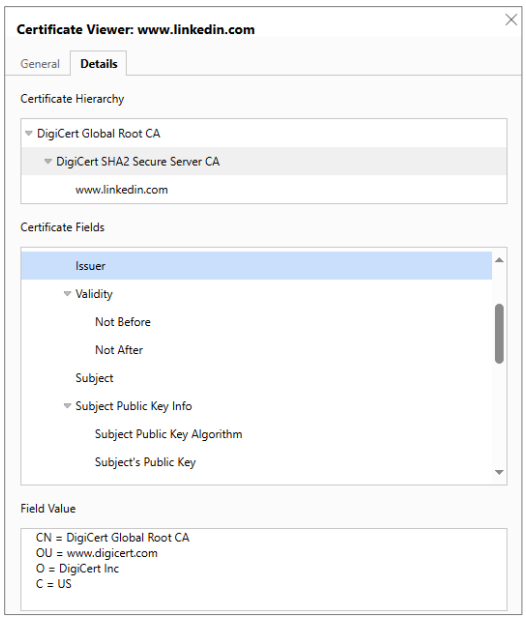
Certificate



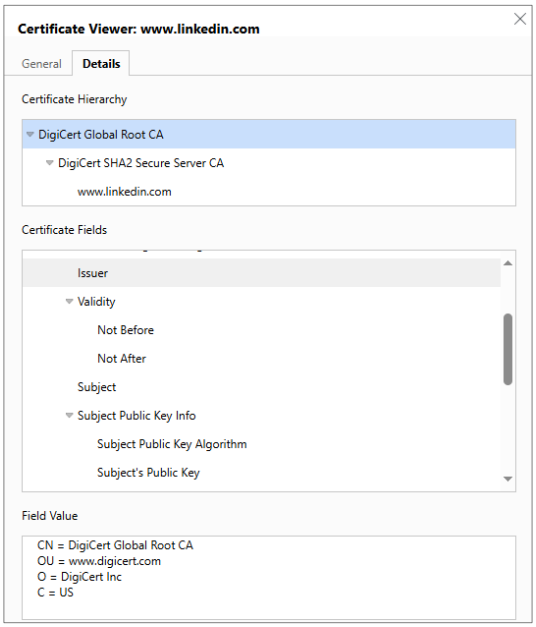


Certificate





Certificate



How does a website obtain a TLS/SSL certificate?

- Certificates come in different types based on the level of validation required:
  - Domain Validation (DV): Verifies domain ownership (basic and fast).
    - ✓ Adding a DNS record.
    - ✓ Uploading a specific file to the website.
  - Organization Validation (OV): Verifies domain ownership and organization details.
  - Extended Validation (EV): Provides the highest level of trust with extensive verification (e.g., for e-commerce or financial sites).
- The applicant generates a certificate signing request (CSR) and submits it to a CA. Once validated, the CA issues the SSL/TLS certificate.
  - CSR includes domain name, organization name, and **public key**.